

STAGE 0-4 / RESEARCH DECISION MEMO

Code Agent KVCache 研究项目

调研结论评审与研究问题收敛

从“广义前缀缓存+通用分层存储”收敛到可验证的单机研究主线

本报告用于 0-4 阶段评审：复核既有 76 篇资料，补入 15 项 2025-2026 年直接相关工作，识别已拥挤方向，形成研究空白、候选路线评分、核心研究问题、可证伪假设和下一阶段门控。报告不是系统实现说明，也不代表 0-4 已经自动通过。

字段	内容
项目阶段	0-4 调研结论评审与研究问题收敛
当前状态	候选完成，待用户评审
资料截止	2026-08-31
证据规模	91 项：既有 76 项+补充 15 项；高相关 36 项
实验环境假设	Ubuntu；NVIDIA GeForce RTX 5090 32GB GDDR7；系统内存 64GB；NVMe 规格待确认
项目状态	立项/申请状态暂定，待项目方确认

待评审结论 将宽泛的“Code Agent 前缀缓存+通用分层存储”收窄；精确前缀为首期正确性边界，非前缀复用仅作高风险拓展。

评审状态： ☐ 认可 ☐ 条件认可 ☐ 要求重构

执行结论

核心判断 项目确实需要收回调研主线。最新证据表明，单纯追求更高前缀命中率、通用多层 KV 缓存、固定 TTL、宽泛 Agent 感知调度或语义淘汰，都已存在直接相邻工作。继续沿原表述推进，会在论文新颖性和项目验收解释上同时遇到风险。

推荐主线 受限单机环境下、面向 Code Agent 暂停/恢复轨迹的有效复用边界与价值密度驱动的分层 KV 保留策略。首期只研究精确 Token 前缀，在 GPU、DRAM、NVMe 和重算之间做收益决策；非前缀复用保留为高风险拓展。

Effective-reuse frontier and value-density-driven tiered KV retention for paused/resumed coding-agent sessions on memory-constrained single-node systems

- 95%原始 Token 命中率降为描述性目标。TraceLab 等工作已经展示高理论复用；本项目要解释的是为何真实系统仍会失效、迁移或负收益。
- “命中”必须增加收益判定。查询、排队、传输与恢复不快于重算时，只能记为逻辑命中，不能记为有效命中。
- 终局指标改为每个成功任务的端到端成本；同时保留 P50/P95/P99 TTFT、会话完成时间、数据移动量和任务正确性。
- 硬件边界固定为 Ubuntu、RTX 5090 32GB GDDR7、64GB 系统内存和本地 NVMe；这不是限制，而是研究问题成立的条件。
- 0-4 仍处于候选完成状态。用户认可主线、RQ、强基线和指标后，才进入 0-5 Trace 与实验方案设计。

一页式决策表

原方向	证据后的判断	0-4 处理
Code Agent 达到 95%前缀命中	高理论命中已被真实轨迹观察；缺口在有效复用	降为上界/描述指标
通用 GPU/DRAM/NVMe 分层	Strata、Tutti、KVDrive、HyMCache 等已覆盖	不单独作为创新
Agent 感知调度与淘汰	CacheWise、Continuum、ThunderAgent、CONCUR、SAECache 高度相邻	全部列为强基线
Trace Replay	TraceLab 和 XPerf 已有数据与重放框架	作为必要基础设施
单机有效复用边界+价值密度分层	现有证据未直接覆盖 5090/64GB/NVMe 四选一边界	推荐主线
仓库来源/依赖感知非前缀复用	潜在新颖，但正确性与安全风险高	仅作拓展 RQ5

1. 阶段定位与评审目标

1.1 当前位于项目 0-4 阶段

0-4 的任务不是选一个框架开始编码，而是把调研结果变成能够指导后续实验的研究决策。上一阶段解决“有哪些资料”；本阶段解决“已有工作做到哪里、项目还能研究什么、什么证据会推翻我们的判断”。

阶段输入	0-4 处理	阶段输出	通过条件
76 篇主题资料、IEEE 英文调研报告、项目定义与硬件约束	补检近期直接竞争工作；证据矩阵；空白与路线评分；形成 RQ/H	Excel 证据矩阵；本评审报告；待确认的研究边界	用户确认推荐主线、RQ1-RQ4、强基线、终局指标和首期范围

1.2 评审问题

- 广义“Code Agent KVCache 系统”是否仍有足够的新颖性？
- 哪些方向只能做基线，哪些方向可成为工程贡献或论文贡献？
- 当前 RTX 5090+64GB 主机能否支撑一个完整、可复现、可证伪的研究闭环？
- 原 70%/95%/90%/10%目标应如何改写，才不会把逻辑命中当成系统收益？
- 什么条件下项目应该停止、调整或降级，而不是继续堆叠功能？

2. 证据范围与方法

2.1 证据规模

证据类别	数量	对 0-4 的作用
Code Agent 工作负载与基准	20	定义任务、轨迹和上下文增长
前缀缓存与上下文复用	14	确定精确/非前缀复用的技术边界
分层存储与解耦推理	12	识别通用分层方向的拥挤程度

证据类别	数量	对 0-4 的作用
压缩淘汰与注意力架构	21	区分容量优化、结构兼容与跨请求复用
评测方法与安全隔离	10	建立正确性、隐私和侧信道边界
近期 Agent 推理与缓存研究	10	核验直接竞争与最新系统主线
成本指标与缓存经济性	3	修正项目终局成本口径
Agent 推理评测与重放	1	判断 Trace Replay 的贡献边界

证据矩阵保留每项工作的资料来源、方法族、工作负载、复用范围、存储层级、评价证据、项目局限、研究空白标签、相关度和置信度。Excel 是可筛选的主证据表，本报告只呈现收敛结论。

2.2 判断规则

规则	说明
主来源优先	论文官方页面、作者项目仓库和厂商官方规格优先；二手材料不承担关键结论。
对象区分	KV 张量缓存、Provider Prompt Cache、中间结果缓存和 Trace 工具不混为同一类贡献。
理论与有效分开	理论可复用前缀不等于实际驻留，也不等于恢复后端到端获益。
高命中不自动等于新颖	若已有工作达到相近命中，项目必须提出新的系统边界、决策问题或评价闭环。
安全和正确性前置	跨租户、非前缀与低精度复用必须先证明兼容性和输出质量。

3. 新增关键证据改变了什么

证据	公开结论	对本项目的影响
TraceLab [1]	真实 Claude Code/Codex 轨迹显示长循环、长上下文和高但不完美的前缀命中	原 95%目标不再足够；转向暂停后的有效复用
CacheWise [2]	已实现前缀感知调度和工具元数据驱动淘汰	宽泛 Agent 感知淘汰不能再作为主创新
Continuum [3]	以 TTL 保留工具停顿中的 KV，并联合重载、重算和排队成本	固定/预测 TTL 进入强基线
ThunderAgent [4]	程序感知地统一 KV 与工具资源调度	程序级调度主线已被覆盖

证据	公开结论	对本项目的影响
Pythia [5]	多 Agent 专用 Prompt 与工作流可出现低命中和资源争用	首期明确限定单 Agent/独立会话
CONCUR [7]	容量未完全耗尽前也会发生中阶段缓存抖动	并发和排队必须进入有效命中条件
SAECache [13]	不同 Token 类型复用价值差异巨大，语义淘汰已有自适应方案	语义信号仅作基线/消融
XPerf [12]	提供 Agent Trace 采集、合成与细粒度重放	Replay 是支撑贡献，不是论文主线
成本研究 [9][10]	Token 减少、缓存计数、账单成本和成功率可能背离	终局指标改为每成功任务成本
HyMCache [15]	CXL 混合内存与 SSD 已覆盖 TB 级多轮 KV 复用	本项目必须突出消费级单机而非泛化分层

最重要的差别 我们不再把“有没有相同前缀”当作最终问题，而是研究同一前缀在 32GB 显存压力、停顿、并发和跨层迁移下是否仍值得保留，以及应该保留在哪一层。

4. 研究空白评审

ID	研究空白	新颖	必要	可行	风险	0-4 决策
G1	原始 Token 命中率不再足以构成主要贡献	1/5	5/5	5/5	低	降为描述性指标；不能作为论文主张
G2	理想命中与真实有效复用之间的鸿沟	4/5	5/5	5/5	中	纳入核心研究问题 RQ1/RQ2
G3	受限单机上 GPU/DRAM/NVMe/重算的联合决策	4/5	5/5	4/5	中	推荐主线；以价值密度和交叉成本为核心
G4	从缓存指标到每个成功任务成本的端到端闭环	4/5	5/5	4/5	中	作为评价贡献和终局验收指标
G5	Prompt 规范化、布局与 Token 语义价值	2/5	3/5	5/5	低	仅作为辅助策略和消融，不作主创新
G6	代码局部变化后的仓库来源/依赖感知非前缀安全复用	5/5	4/5	2/5	高	高风险拓展项；不进入首期核心承诺
G7	多租户隔离、模型版本与缓存键安全边界	2/5	5/5	4/5	高	作为强制边界与负面测试，不作为主要性能创新
G8	单 Agent 与多 Agent、GQA 与 MLA 等适用范围差异	2/5	5/5	4/5	中	第一阶段限定单 Agent、GQA、精确前缀；其余延后

4.1 推荐空白：理想到有效复用的鸿沟

理论 Token 命中只要求当前 Token 前缀在历史中存在；有效复用还要求缓存仍驻留、兼容域一致、查询和恢复成功，并且完整恢复路径快于重算。Code Agent 的工具等待、人类确认、长尾命令和多会话竞争使这两者产生结构性差距。

$$T_{\text{lookup}} + T_{\text{queue}} + T_{\text{transfer}} + T_{\text{restore}} < T_{\text{recompute}}$$

只有满足上式的命中 Token 才进入有效 Token 命中率分子；否则记为逻辑命中或负收益命中。

4.2 推荐空白：受限单机的四选一层级决策

对每个暂停会话或 KV 块，候选动作是保留 GPU、迁移 DRAM、迁移 NVMe 或直接逐出并在返回时重算。现有工作提供了 TTL、预测、通用分层和专用硬件方案，但对 RTX 5090 32GB+64GB DRAM 的联合交叉面缺少直接验证。

$$Benefit(i,l) = p_{return}(i) \cdot [T_{recompute}(i) - T_{restore}(i,l)] - C_{move}(i,l) - \lambda_l \cdot Bytes(i)$$

该式是待实验验证的候选决策框架，不是已完成算法。若所有层级收益均不为正，则选择重算。容量约束下优先保留单位字节收益更高的条目。

4.3 高风险空白：代码来源/依赖感知非前缀复用

同一段代码文本在不同前置上下文或位置上通常不具有相同 KV 状态。CacheBlend、EPIC、Irminsul 等已研究选择性重算或位置无关复用，同时安全文献暴露了侧信道、反演和劫持风险。因此 RQ5 只能在精确前缀主线完成后启动，且任何近似都必须通过 Logits、Token 和任务成功三级门槛。

5. 候选路线评分与选择

路线	方向	加权分	决策	理由
A	继续追求原始前缀 Token 命中率	3.40	不作为主线	已有真实轨迹接近或超过原 95%目标；能做基线但难形成新贡献。
B	通用 GPU/DRAM/NVMe 多层 KV 缓存	3.00	不单独作为主线	Strata、Tutti、KVDrive、HyMCache 等已占据通用分层方向。
C	宽泛 Agent 感知调度/TTL/语义淘汰	3.00	仅作强基线	CacheWise、Continuum、ThunderAgent、CONCUR、SAECache 覆盖度高。
D	受限单机的有效复用边界与价值密度分层保留	4.10	推荐主线	面向 RTX 5090+64GB DRAM+NVMe，联合决定 GPU 保留、下沉或重算，并用有效命中与成功任务成本评估。
E	仓库来源/依赖感知的非前缀 KV 安全复用	3.25	高风险拓展	潜在创新较强，但 Transformer 依赖、位置与攻击面使正确性风险很高。
F	仅构建 Agent Trace Replay 基准	3.40	支撑工具，不作主线	TraceLab 和 XPerf 已提供相近能力；仍是实验可复现的必要基础设施。

评分权重：新颖性 40%、可行性 20%、证据支撑 15%、范围可控 15%、安全性 10%。加权分只辅助决策：A 和 F 虽可行，但新颖性不足；E 虽新颖，但首期正确性风险不可接受。

路线 D 获选 受限单机的有效复用边界与价值密度分层保留获得 4.10/5，既能使用现有 5090/64GB 环境完成，又与通用分层、TTL 和 Agent 调度形成可解释差异。

6. 推荐研究主线的精确定义

6.1 研究对象

研究暂停/恢复型 Coding Agent 会话：一次任务由多次 LLM 调用与工具执行交替组成，后续调用通常追加少量 Token，但返回时间不确定。系统在每次暂停时决定 KV 状态的驻留层级，在会话恢复时决定加载或重算。

边界	首期定义
部署	单节点 Ubuntu；RTX 5090 32GB；系统内存 64GB；本地 NVMe 规格待确认
工作负载	单 Agent 或独立会话的 Coding Agent；多 Agent 仅作外推风险讨论
模型	先选一个 GQA 模型；BF16/FP16；模型、Tokenizer、RoPE 和引擎版本进入缓存键
复用	精确 Token 前缀；只复用严格兼容的 KV；不按文本相似度直接复用
动作	GPU 保留 / DRAM 迁移 / NVMe 迁移 / 逐出后重算
目标	最大化有效复用和会话级收益，同时控制数据移动、容量与任务正确性

6.2 系统输入与决策变量

信号	示例字段	作用
返回时间	工具类型、历史时长、是否等待人类确认、会话状态	估计 p_return 和可容忍驻留时间
计算价值	前缀 Token 数、模型/层、实测 Prefill 时间	估计重算成本
迁移成本	KV 字节、层级带宽、排队、反序列化	判断恢复是否快于重算
容量压力	GPU/DRAM/NVMe 占用、活跃会话、预期新请求	形成单位字节价值和准入

信号	示例字段	作用
安全兼容	tenant、project、model、tokenizer、layout、dtype	决定是否允许查询和复用

6.3 项目贡献拆分

- 测量贡献：量化理论命中、实际驻留命中与有效收益之间的差距，并给出按停顿、长度、并发和层级划分的边界图。
- 系统贡献：在受限单机上实现 GPU/DRAM/NVMe/重算的可审计联合决策，并提供强基线与完整消融。
- 评价贡献：把原始命中、负收益命中、数据移动和每成功任务成本放在同一端到端框架中。
- 工程贡献：沉淀可重复的 Trace、微基准、事件日志、配置快照和一键实验脚本；但不把 Replay 本身包装为首创。

7. 研究问题与可证伪假设

ID	研究问题	方法	退出标准
RQ1	在 RTX 5090 32GB 显存约束下，Code Agent 理论前缀复用率与真实有效复用率之间有多大差距？	Trace 重放+可控合成轨迹；逐层记录查询、传输、恢复、重算时间	给出按负载分区的有效复用边界图，并能解释主要失效来源
RQ2	暂停会话何时应留在 GPU、迁移 DRAM、迁移 NVMe，或直接重算？	微基准拟合 load-vs-recompute 交叉面；与固定阈值和固定 TTL 比较	形成可测量的四选一决策规则，且不依赖单一固定阈值
RQ3	价值密度策略能否在相同容量下优于 LRU、固定 TTL 和 DRAM-only 基线？	等容量、等 Trace、等模型对比；完整消融返回时间、重算成本和字节密度信号	至少在两个真实/近真实负载区间稳定优于强基线，而非只在单一合成点获益
RQ4	缓存收益在加入任务成功率与端到端资源成本后是否仍然成立？	固定模型、Tokenizer、Prompt 与 Harness；对成功任务进行成本归一化	性能结论与成功任务成本方向一致；若背离则明确否定原假设

ID	研究问题	方法	退出标准
RQ5（拓展）	仓库来源和依赖信息能否为代码局部修改后的非前缀复用提供安全约束？	仅在精确前缀主线完成后，以选择性重计算和严格输出校验开展	任何近似复用都必须通过质量门槛；不满足即停止该路线

7.1 核心 RQ

RQ1 在 RTX 5090 32GB 显存约束下，Code Agent 理论前缀复用率与真实有效复用率之间有多大差距？

压力维度：会话间隔、并发数、上下文长度、缓存预算、修改位置。指标：理论 Token 命中率、有效 Token 命中率、逐出率、Prefill 放大、TTFT。

RQ2 暂停会话何时应留在 GPU、迁移 DRAM、迁移 NVMe，或直接重算？

压力维度：预计返回时间、KV 字节数、前缀长度、层级带宽/延迟、GPU 排队压力。指标：迁移时间、重算时间、GPU 占用、数据移动量、恢复 TTFT。

RQ3 价值密度策略能否在相同容量下优于 LRU、固定 TTL 和 DRAM-only 基线？

压力维度：策略、缓存容量、并发、停顿分布、工具类型。指标：有效命中、P50/P95/P99 TTFT、会话完成时间、吞吐、迁移字节。

RQ4 缓存收益在加入任务成功率与端到端资源成本后是否仍然成立？

压力维度：缓存策略、Prompt/Harness 固定与变化、任务难度。指标：成功率、代码测试通过率、每成功任务 GPU 秒/能耗/成本、输出一致性。

7.2 可证伪假设

ID	假设	验证	证伪条件
H1	追加型 Code Agent 轨迹的理论前缀复用很高，但 32GB 显存、暂停与并发会显著降低有效复用。	比较无限缓存理论值、GPU 受限值和分层恢复后的有效值。	三者差异在所有代表性负载下均很小。
H2	load-vs-recompute 交叉点同时依赖前缀长度、层级带宽和排队压力，单一固定阈值无法覆盖。	对 GPU/DRAM/NVMe 分别拟合交叉面，并跨负载验证。	固定阈值在所有负载上与拟合策略等效。
H3	联合返回概率、重算成本和字节占用的价值密度策略，在等容量下优于 LRU 和固定 TTL。	与 LRU、固定 TTL、CacheWise 式预测和 DRAM-only 基线做等容量对比。	强基线在主要指标上持平或更优。

ID	假设	验证	证伪条件
H4	优化原始命中率可能增加跨层数据移动或排队，因而降低吞吐；有效命中率与端到端收益相关性更高。	同时记录原始命中、有效命中、数据移动、TTFT 和吞吐并做相关分析。	原始命中率始终能可靠预测端到端收益。
H5	分层保留的性能增益在任务正确性与每成功任务成本校正后仍然存在。	运行代码任务测试并按成功任务归一化资源成本。	策略提高缓存指标但降低成功率或成功任务成本不降。
H6（拓展）	仓库来源/依赖约束可安全恢复部分非前缀机会，但必须依赖选择性重算和严格校验。	局部修改合成实验+Logits/Token/任务三级正确性门槛。	误差或失败率不可接受，或收益不足以覆盖修复成本。

研究纪律 若强基线在主要负载上持平或更优，或任务成功率/每成功任务成本恶化，则必须否定相应假设，而不是只展示有利切片。

8. 基线与指标冻结建议

8.1 强制基线

ID	基线	作用	强制
B0	无跨请求缓存	端到端重算基线	是
B1	vLLM 精确前缀+LRU	生产式基础前缀缓存	是
B2	固定 TTL GPU 保留	Continuum/TTL 思想的简化强基线	是
B3	工具时长/返回概率预测淘汰	CacheWise 式 Agent 感知基线	是
B4	DRAM-only 卸载（LMCache 式）	验证单层主机内存迁移收益	是
B5	GPU+DRAM+NVMe 价值密度策略	本项目候选方案	是
B6	Provider Keepalive	仅作 API 经济学分析对照，不与本地系统混为一谈	否

B2 和 B3 不要求完整复刻外部论文，但必须实现同类信息优势：固定 TTL 与工具时长/返回概率预测。B4 用于分离“有无主机内存卸载”与“是否需要 NVMe/价值密度”的贡献。

8.2 指标体系

组别	指标	定义	用途
复用	理论 Token 命中率	可找到相同 Token 前缀的 Token 数 / 全部 Prefill Token 数	描述性上界
复用	有效 Token 命中率	满足查询+传输+恢复 < 重算的复用 Token 数 / 全部 Prefill Token 数	主指标
延迟	TTFT P50/P95/P99	端到端请求到首 Token 延迟，区分完全命中/部分命中/未命中	主指标
会话	Session Completion Time	从任务开始到 Agent 任务终止的墙钟时间	主指标
资源	KV 数据移动量	GPU ↔ DRAM、DRAM ↔ NVM _e 的读写字节及时间	机制解释
资源	GPU 秒与能耗	每请求/每会话 GPU 活动时间与功耗积分	成本输入
正确性	输出与任务成功	Logits/Token 一致性、测试通过率与任务成功率	硬门槛
终局	每成功任务成本	端到端 GPU、CPU、存储、能耗成本 / 成功任务数	终局指标

8.3 三个必须冻结的公式

$$H_{\text{token}} = \Sigma \text{ reused prefill tokens} / \Sigma \text{ all prefill tokens}$$

$$H_{\text{effective}} = \Sigma \text{ reused tokens satisfying } T_{\text{restore-path}} < T_{\text{recompute}} / \Sigma \text{ all prefill tokens}$$

$$C_{\text{success}} = \text{total end-to-end system cost} / \text{number of successful tasks}$$

原 70%、95%、TTFT 下降 90%和成本 10%以下可继续作为立项候选目标，但必须同时绑定 Trace 版本、缓存预算、冷/热规则、并发、模型、成功率和统计窗口；在测得单机基线前不能写成已证明事实。

9. 首期范围与明确排除项

纳入首期	暂不纳入首期
• 单节点 Ubuntu, RTX 5090 32GB 显存、64GB DRAM 与本地 NVMe • 单 Agent 或独立会话的暂停/恢复型 Coding Agent 轨迹 • GQA 模型、BF16/FP16、精确 Token 前缀复用 • GPU/DRAM/NVMe/重算四选一保留与恢复决策 • Trace 重放、微基准、正确性、端到端性能与成功任务成本	• 首期不承诺 MLA/DSA/Hybrid 和 FP4/FP8 全兼容 • 首期不承诺跨租户共享与模糊文本相似度直接复用 • 首期不承诺多节点、CXL 或远端对象存储 • 首期不把非前缀/位置无关复用作为验收硬目标 • 不把 95%原始命中率写成已验证成果或唯一成功标准

安全边界 租户、项目、模型、Tokenizer、位置编码、KV 布局、数值格式和引擎版本必须进入兼容域。跨租户私有内容共享和任意文本相似度复用默认禁止。

10. 在现有硬件上的可行性与限制

10.1 为什么 RTX 5090 + 64GB 适合该问题

NVIDIA 官方规格显示 GeForce RTX 5090 具有 32GB GDDR7 显存[25]。这足以运行中等规模开源模型并测量长上下文 Prefill，却不足以无限保留多个长会话的 KV，因此能够自然产生 GPU 保留、DRAM 迁移、NVMe 迁移和重算之间的压力。64GB 系统内存提供有限但可用的第二层缓存。

资源	已知条件	研究意义	待确认
GPU	RTX 5090, 32GB GDDR7	形成真实显存压力；测量 Prefill 和恢复	驱动/CUDA、功耗策略
DRAM	64GB	主机 KV 热层；容量仍有限	可分配预算、锁页内存策略
NVMe	本地，型号未知	温层与持久层；决定迁移交叉点	型号、容量、PCIe 代际、实测带宽/IOPS
OS	Ubuntu	支持 vLLM、CUDA 和系统剖析工具	发行版与内核版本
模型/引擎	暂未冻结	决定 KV 字节、支持格式和基线	建议首个 GQA 模型 + vLLM

10.2 首期不承诺的外推

- 不能从单机 5090 直接外推 H100 集群、PD 解耦、CXL 或远端存储的绝对收益。
- 不能从单 Agent 追加轨迹直接外推多 Agent 工作流；Pythia 已显示其前缀结构可能不同。
- 不能从 GQA/BF16 直接外推 MLA、DSA、Hybrid、FP8 或 FP4；KV 布局和质量门槛必须重新验证。
- 不能用 Provider Prompt Cache 的计费行为替代自托管 KV 张量的带宽和容量实验。

11. 实验框架草案（供 0-5 继续细化）

11.1 数据源组合

数据源	用途	0-5 动作
CacheWise 公开 Coding Agent 轨迹	直接竞争基线和工具元数据信号	核验许可、字段、Token 可回放性
TraceLab 公开轨迹/分析代码	真实停顿、工具长尾和高理论命中	下载并建立字段映射
XPerf（若代码已公开）	可重复的 Agent serving 重放	评估复用而非重复造轮子
SWE-bench/OpenHands 近真实轨迹	任务成功率与会话完成时间	固定模型/Harness，保存工具调用
可控合成轨迹	扫描长度、间隔、变化率、并发和容量	形成正交实验矩阵

11.2 最小实验矩阵

因素	建议水平	目的
上下文长度	8K / 32K / 64K / 128K 或按模型能力调整	建立重算成本和 KV 字节曲线
暂停间隔	秒级 / 分钟级 / 人类确认级	测量返回概率和 TTL 失效
并发会话	1 / 2 / 4 / 8 / 显存抖动点	定位中阶段抖动
缓存预算	GPU 可用 KV 预算的 25% / 50% / 75% / 100%	比较等容量策略
层级	GPU / DRAM / NVMe / 重算	拟合交叉面
代码变化	0 / 局部 / 中等 / 大规模	首期测前缀失效；拓展测来源约束

11.3 结果报告

- 每个负载点报告原始 Token 命中、实际驻留命中、有效命中和负收益命中。
- TTFT 同时报告 P50/P95/P99 和完全命中、部分命中、未命中三类切片。
- 记录 GPU ↔ DRAM、DRAM ↔ NVMe 的字节、时间和并发带宽，解释任何反常收益。

- 任务实验报告测试通过率、输出一致性和每成功任务 GPU 秒/能耗/成本。
- 所有结论与强基线等容量、等 Trace、等模型、等 Prompt/Harness 比较。

12. 风险、停止条件与 0-4 退出门

12.1 主要风险

风险	触发信号	处理
研究重叠	CacheWise/Continuum 式基线已覆盖候选方案	进一步收窄到层级交叉面、单机约束和成功任务成本；不夸大新颖性
5090 不支持目标软件路径	引擎/内核在 Blackwell 消费卡上不可用	先冻结支持矩阵；选择稳定 GQA/BF16 路径
NVMe 迁移始终不划算	所有工作点恢复均慢于重算	把负结果写成边界结论；主线可退化为 GPU/DRAM/重算三选一
预测收益不稳定	价值密度策略不优于固定 TTL/LRU	否定 H3；保留测量贡献和交叉面结果
质量或安全回归	输出差异、任务成功率下降或跨域命中	停止对应近似/共享路线；回到精确前缀隔离
Trace 不可回放	缺 Token、输出或工具时序	用可控 Trace 补因果分析，并在报告中限制外推

12.2 0-4 退出条件

1. 用户认可主线收窄与研究边界
2. 用户认可 RQ1-RQ4 为核心，RQ5 仅作拓展
3. 用户认可强基线和终局指标
4. 确认本机 NVMe 型号/容量/顺序读写能力
5. 确认首个模型与推理框架（建议 GQA + vLLM）
6. 评审通过后再归档 GitHub 并进入 0-5 Trace 与实验方案设计

阶段状态 当前为“候选完成，待用户评审”。只有前述核心决策获得认可，才在 GitHub 归档 0-4 成果并进入 0-5；在此之前不开端缓存系统实现。

13. 建议评审批注

评审项	推荐选择	用户意见
主线	认可路线 D：有效复用边界+价值密度分层保留	
首期范围	单 Agent、GQA、BF16/FP16、精确前缀、单机三层	
核心 RQ	RQ1-RQ4；RQ5 仅作拓展	
强基线	B0-B5 全部保留；B6 只作 API 对照	
终局指标	每成功任务成本+任务正确性硬门槛	
下一阶段	0-5 Trace 来源与实验方案设计，不做系统实现	

参考文献与官方资料

- [1] K. Zhu et al., "TraceLab: Characterizing Coding Agent Workloads for LLM Serving," arXiv:2606.30560, 2026.
<https://arxiv.org/abs/2606.30560>
- [2] S. Tiwari et al., "CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents," arXiv:2606.16824, 2026. <https://arxiv.org/abs/2606.16824>
- [3] H. Li et al., "Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live," arXiv:2511.02230, 2025/2026. <https://arxiv.org/abs/2511.02230>
- [4] H. Kang et al., "ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System," arXiv:2602.13692, 2026.
<https://arxiv.org/abs/2602.13692>
- [5] S. Yu et al., "Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving," arXiv:2604.25899, 2026.
<https://arxiv.org/abs/2604.25899>
- [6] Y. Yuan et al., "Agentic AI Workload Characteristics," arXiv:2605.26297, 2026. <https://arxiv.org/abs/2605.26297>
- [7] Q. Chen et al., "CONCUR: High-Throughput Agentic Batch Inference of LLM via Congestion-Based Concurrency Control," arXiv:2601.22705, 2026. <https://arxiv.org/abs/2601.22705>
- [8] Y. Yuan, M. Chowdhury, and N. Talati, "KAIROS: Stateful, Context-Aware Power-Efficient Agentic Inference Serving," arXiv:2604.16682, 2026. <https://arxiv.org/abs/2604.16682>
- [9] S. Weinberger and A. Hozes, "Token Reduction Is Not Cost Reduction," arXiv:2607.12161, 2026. <https://arxiv.org/abs/2607.12161>
- [10] S. Weinberger and A. Hozes, "Prompt-Induced Waste in Coding Agents," arXiv:2608.01347, 2026.
<https://arxiv.org/abs/2608.01347>
- [11] A. Mohamed et al., "Workload-Aware Caching for Multi-Agent Systems," arXiv:2607.20495, 2026.
<https://arxiv.org/abs/2607.20495>
- [12] M. Wang et al., "Benchmarking LLM Serving Systems for Agentic AI Workloads with XPerf," arXiv:2608.20370, 2026.
<https://arxiv.org/abs/2608.20370>
- [13] S. Fang et al., "Not All Tokens Are Worth Caching: Learning Semantic-Aware Eviction for LLM Prefix Caches," arXiv:2605.18825, 2026. <https://arxiv.org/abs/2605.18825>
- [14] M. Khailo, "Keeping the Cache Warm Pays: Keepalive Economics for Agentic Workloads," arXiv:2607.19214, 2026.
<https://arxiv.org/abs/2607.19214>
- [15] H. Jang et al., "HyMCache: A KV Cache Framework for Multi-Turn LLM Serving with CXL-Hybrid Memory," arXiv:2607.18141, 2026.
<https://arxiv.org/abs/2607.18141>
- [16] L. Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs," arXiv:2312.07104.
<https://arxiv.org/abs/2312.07104>
- [17] Y. Yao et al., "CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion," arXiv:2405.16444.
<https://arxiv.org/abs/2405.16444>
- [18] "EPIC: Efficient Position-Independent Caching for Serving Large Language Models," arXiv:2410.15332.
<https://arxiv.org/abs/2410.15332>
- [19] "Compute Or Load KV Cache? Why Not Both?" arXiv:2410.03065. <https://arxiv.org/abs/2410.03065>
- [20] "LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference," arXiv:2510.09665. <https://arxiv.org/abs/2510.09665>
- [21] "Strata: Hierarchical Context Caching for Long Context Language Model Serving," arXiv:2508.18572.
<https://arxiv.org/abs/2508.18572>
- [22] "KVDrive: A Holistic Multi-Tier KV Cache Management System for Long-Context LLM Inference," arXiv:2605.18071.
<https://arxiv.org/abs/2605.18071>
- [23] "Rethinking Key-Value Cache Compression Techniques for Large Language Model Serving," arXiv:2503.24000.
<https://arxiv.org/abs/2503.24000>
- [24] "Auditing Prompt Caching in Language Model APIs," arXiv:2502.07776. <https://arxiv.org/abs/2502.07776>
- [25] NVIDIA, "GeForce RTX 5090 Specifications," 32 GB GDDR7.
<https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5090/>

附录 A：15 项补充文献索引

ID	年	题目	Gap	官方入口
S01	2026	TraceLab: Characterizing Coding Agent Workloads for LLM Serving	G2 有效复用鸿沟	https://arxiv.org/abs/2606.30560
S02	2026	CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents	G3 联合层级决策	https://arxiv.org/abs/2606.16824
S03	2025	Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live	G3 联合层级决策	https://arxiv.org/abs/2511.02230
S04	2026	ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System	G2 有效复用鸿沟	https://arxiv.org/abs/2602.13692
S05	2026	Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving	G8 适用边界	https://arxiv.org/abs/2604.25899
S06	2026	Agentic AI Workload Characteristics	G2 有效复用鸿沟	https://arxiv.org/abs/2605.26297
S07	2026	CONCUR: High-Throughput Agentic Batch Inference of LLM via Congestion-Based Concurrency Control	G2 有效复用鸿沟	https://arxiv.org/abs/2601.22705
S08	2026	KAIROS: Stateful, Context-Aware Power-Efficient Agentic Inference Serving	G4 成功任务成本	https://arxiv.org/abs/2604.16682
S09	2026	Token Reduction Is Not Cost Reduction	G4 成功任务成本	https://arxiv.org/abs/2607.12161
S10	2026	Prompt-Induced Waste in Coding Agents: Reasoning, Effort, Harness Design, and End-to-End Cost	G4 成功任务成本	https://arxiv.org/abs/2608.01347
S11	2026	Workload-Aware Caching for Multi-Agent Systems	G3 联合层级决策	https://arxiv.org/abs/2607.20495
S12	2026	Benchmarking LLM Serving Systems for Agentic AI Workloads with XPerf	G8 适用边界	https://arxiv.org/abs/2608.20370
S13	2026	Not All Tokens Are Worth Caching: Learning Semantic-Aware Eviction for LLM Prefix Caches	G5 Prompt 与语义布局	https://arxiv.org/abs/2605.18825
S14	2026	Keeping the Cache Warm Pays: Keepalive Economics for Agentic Workloads	G4 成功任务成本	https://arxiv.org/abs/2607.19214
S15	2026	HyMCache: A KV Cache Framework for Multi-Turn LLM Serving with CXL-Hybrid Memory	G3 联合层级决策	https://arxiv.org/abs/2607.18141

附录 B：与 Excel 证据矩阵的关系

配套工作簿《Code_Agent_KVCache_Stage_0-4_Evidence_Gap_Matrix_v1.xlsx》包含 91 项完整证据矩阵、8 项研究空白、6 条候选路线评分、5 个研究问题、6 项假设、强制基线、指标口径和 15 项补充文献。报告结论与工作簿字段一一对应；后续更新应先改证据矩阵，再更新本报告。

附录 C：文档控制与变更规则

- 0-4 评审通过前，报告、工作簿和 GitHub 版本均保持“候选完成，待用户评审”状态。
- 新增直接竞争论文时，先更新证据矩阵的项目局限和 Gap 标签，再重新计算候选路线评分。
- 若首期模型、推理框架或 NVMe 硬件改变，应在 0-5 实验方案中更新交叉成本假设，不回写为既成结论。
- 任何对外论文、专利或项目申请必须把候选目标与实测结果分开，并标注数据版本和统计口径。

下一步 用户评审通过后，进入 0-5 阶段：冻结 Trace 来源、模型/引擎、NVMe 微基准和实验矩阵；仍不直接开始完整缓存系统实现。